

Backtracking Problems



Agencia



1. N-Queens
2. Sudoku Solver
3. Word Break Problems

Hello Everyone

Very Special Good Evening

to all of you 😊😊😊

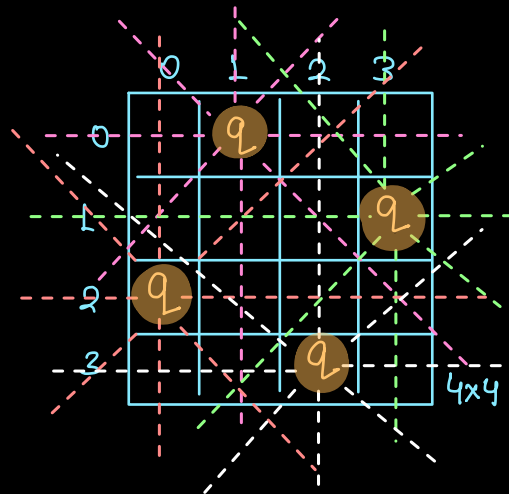
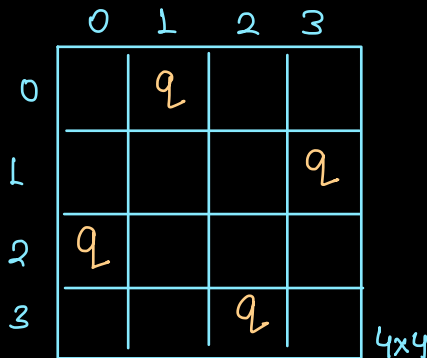
We will start session

from 09:06 PM

N-Queens

Given $N \times N$ chess board, Place N Queens in these chess board and possible arrangement.

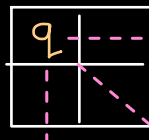
$N=4$



$N=1$

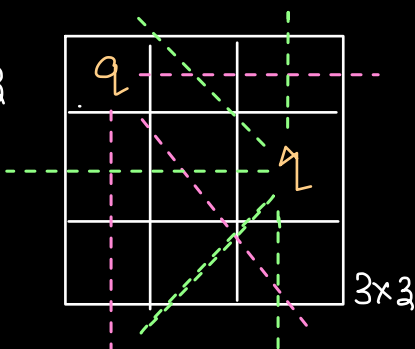


, $N=2$



not possible

$n=3$



NOTE: for n-queen problem, value of n must be: $n \geq 4$

$N=4$

	0	1	2	3
0		q		
1				q
2	q			
3			q	

4x4

	0	1	2	3
0			q	
1	q			
2				q
3		q		

4x4

Observations:

#1. Each row contains exactly one queen.

#2. One queen have n -cell options are there.

$N=4$
=

$i \rightarrow$

	0	1	2	3
0				
1				
2				
3				

-	q	-	-
-	-	-	q
q	-	-	-
-	-	q	-

-	-	q	-
q	-	-	-
-	-	-	q
-	q	-	-

Recursive Syntax:

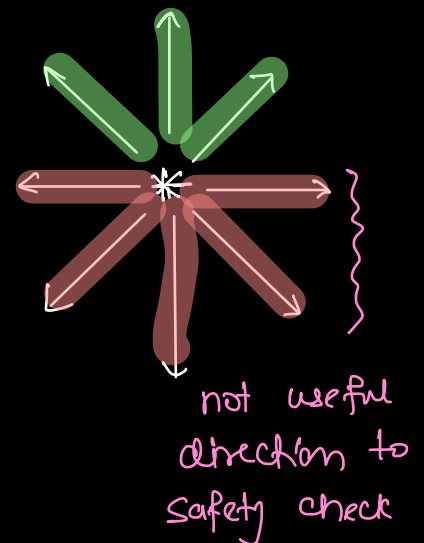
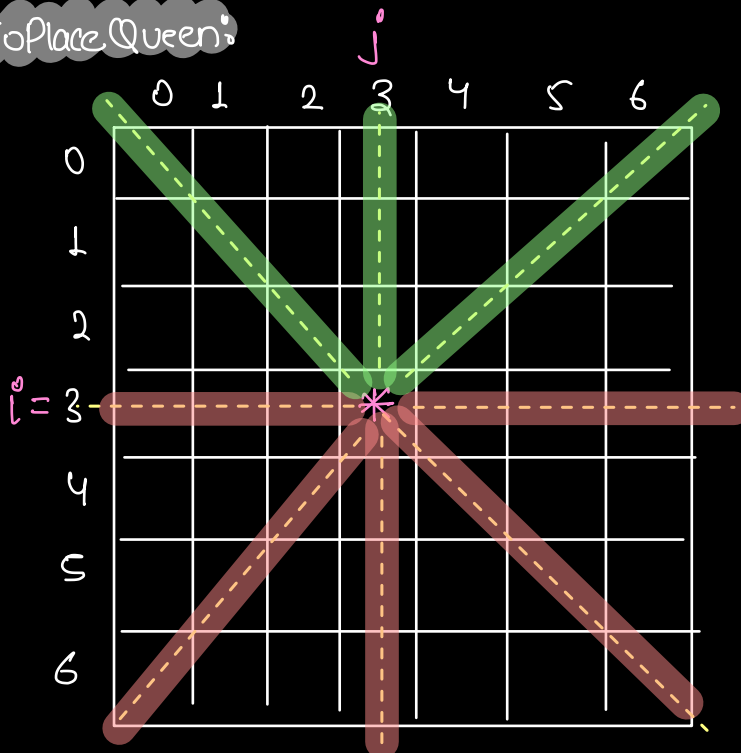
board $\rightarrow$ char[][]
Row No $\rightarrow$ i

Code:

```
/*
 * 1. We have to place one queen in every row
 * 2. Each queen have n options in every row
 */
public static void n_queens(char[][] board, int i) {
    // base case
    if(i == board.length) {
        // all n queens are placed in each row
        printBoard(board);
        return;
    }

    // for ith row, we can iterate in it and try each cell
    for(int j = 0; j < board.length; j++) {
        if(isSafeToPlaceQueen(board, i, j)) {
            // place
            board[i][j] = 'Q';
            // make call for next row
            n_queens(board, i + 1);
            // unplace -> backtracking
            board[i][j] = '-';
        }
    }
}
```

is Safe To Place Queen:



not useful → Red-Region

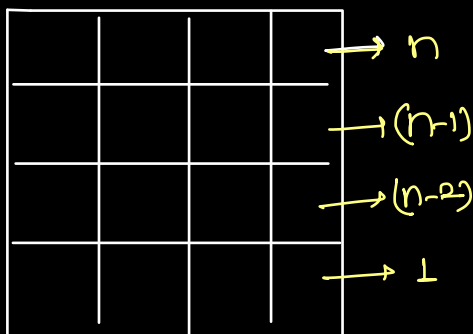
Because when we are placing a queen in (i, j) location, no queen placed in same Row & in down Rows.

```

public static boolean isSafeToPlaceQueen(char[][] board, int i, int j) {
    // we need to check top direction, left-top diagonal & right-top diagonal
    // top direction
    for(int r = i, c = j; r >= 0 ; r--) {
        if(board[r][c] != '-') return false;
    }
    // top-left diagonal
    for(int r = i, c = j; r >= 0 && c >= 0; r--, c--) {
        if(board[r][c] != '-') return false;
    }
    // top-right diagonal
    for(int r = i, c = j; r >= 0 && c < board.length ; r--, c++) {
        if(board[r][c] != '-') return false;
    }
    return true;
}

public static void printBoard(char[][] board) {
    int n = board.length;
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            System.out.print(board[i][j] + " ");
        }
        System.out.println();
    }
    System.out.println();
}

```



$$\text{total options} \Rightarrow n * (n-1) * (n-2) * \dots * 1$$

$$= n!$$

$$\Rightarrow \text{T.C: } O(n!)$$

$$\text{S.C} = O(n)$$

Sudoku Solver

You are given an 9×9 board, in which the cells contains numbers from 1 - 9. You need to check the below conditions,

1. Each row must contain the numbers from 1 to 9 w/o repetitions.
2. Each col must contain the numbers from 1 to 9 w/o repetitions.
3. Each block of size $3 * 3$ should contain all numbers from 1 to 9 w/o repetition.

	0	1	2	3	4	5	6	7	8
0									
1									
2									
3									
4									
5									
6									
7									
8									

Input:

	0	1	2	3	4	5	6	7	8
0	5	3	0	0	7	0	0	0	0
1	6	0	0	1	9	5	0	0	0
2	0	9	8	0	0	0	0	6	0
3	8	0	0	0	6	0	0	0	3
4	4	0	0	8	0	3	0	0	1
5	7	0	0	0	2	0	0	0	6
6	0	6	0	0	0	0	2	8	0
7	0	0	0	4	1	9	0	0	5
8	0	0	0	0	8	0	0	7	9

$\text{board}[i][j] == 0$
An empty cell

$i=0, j=2$

Options for this
cell $\rightarrow 1$ to 9

	0	1	2	3	4	5	6	7	8
0	5	3	0	0	7	0	0	0	0
1	6	0	0	1	9	5	0	0	0
2	0	9	8	0	0	0	0	6	0
3	8	0	0	0	6	0	0	0	3
4	4	0	0	8	0	3	0	0	1
5	7	0	0	0	2	0	0	0	6
6	0	6	0	0	0	0	2	8	0
7	0	0	0	4	1	9	0	0	5
8	0	0	0	0	8	0	0	7	9

make a pair array & store position of empty cell in it.

0,2 | 0,3 | 0,5 | 0,6 | 0,7...-----

all empty cells are available in an array.

↑
explore each empty cell with options from 1 to 9 & move on next empty cell.

	0	1	2	3	4	5	6	7	8
0									
1									
2									
3									
4						*			
5									
6									
7									
8									

→ entire Row

$i=4$
 $j=5$

Start Row = 3
Start Col = 3

Start Row = $i - i \% 3$

Start Col = $j - j \% 3$

Start Row = $i - i \% 3$

Start Col = $j - j \% 3$

$i=0, j=8$

$$r = 0 - 0 \% 3 = 0$$

$$c = 8 - 8 \% 3 = 6$$

$(0, 6) \checkmark$

$i=8, j=2$

$$r = 8 - 8 \% 3 = 6$$

$$c = 2 - 2 \% 3 = 2$$

$(6, 2) \checkmark$

$i=5, j=7$

$$r = 5 - 5 \% 3 = 2$$

$$c = 7 - 7 \% 3 = 6$$

$(2, 6) \checkmark$

$i=5, j=1$

$$r = 5 - 5 \% 3 = 2$$

$$c = 1 - 1 \% 3 = 1$$

$(2, 1) \checkmark$

$i=6, j=2$

$$r = 6 - 6 \% 3 = 6$$

$$c = 2 - 2 \% 3 = 2$$

$(6, 2) \checkmark$

$i=8, j=8$

$$r = 8 - 8 \% 3 = 6$$

$$c = 8 - 8 \% 3 = 6$$

$(6, 6) \checkmark$

We will move toward code after

→ 10:25 - 10:25 PM
Break

Output:

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

```
// ===== Sudoku Problem =====
public static class Pair {
    int x;
    int y;
    Pair(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

public static void sudoku(int[][] board) {
    List<Pair> list = new ArrayList<>();
    // iterate on board and store empty cell in pair array
    for(int i = 0; i < 9; i++) {
        for(int j = 0; j < 9; j++) {
            if(board[i][j] == 0) {
                // empty location, store it in pair's list
                list.add(new Pair(i, j));
            }
        }
    }
    solveSudoku(board, list, indx:0);
}

/*
 * Funfact abotu sudoku
 * If we want unique solution in sudoku, we have to fill atleast 17
 * cells in it.
 */
```

```
public static void solveSudoku(int[][] board, List<Pair> list, int indx) {
    if(indx == list.size()) {
        // all locqation is solved, print the board
        printSudokuBoard(board);
        return;
    }
    int r = list.get(indx).x;
    int c = list.get(indx).y;
    for(int num = 1; num <= 9; num++) {
        if(isSafeToPlaceInSudoku(board, r, c, num)) {
            // place
            board[r][c] = num;
            // call for next unsolved location
            solveSudoku(board, list, indx + 1);
            // unplace
            board[r][c] = 0;
        }
    }
}

}
```



```

public static boolean isSafeToPlaceInSudoku(int[][] board,
    int r, int c, int num) {
    // check in same row
    for(int i = r, j = 0; j < 9; j++) {
        if(board[i][j] == num) return false;
    }
    // check in same col
    for(int i = 0, j = c; i < 9; i++) {
        if(board[i][j] == num) return false;
    }
    // check in sub matrix
    // find starting row and column
    r = r - r % 3;
    c = c - c % 3;
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            if(board[i + r][j + c] == num) return false;
        }
    }
    return true;
}

public static void printSudokuBoard(int[][] board) {
    for(int i = 0; i < 9; i++) {
        for(int j = 0; j < 9; j++) {
            System.out.print(board[i][j] + " ");
        }
        System.out.println();
    }
    System.out.println();
}

```



```
// ===== Word Break =====
public static void solveWordBreak(HashSet<String> dict,
    String word, String ans) {
    if(word.length() == 0) {
        System.out.println(ans);
        return;
    }

    for(int i = 0; i < word.length(); i++) {
        String substr = word.substring(beginIndex:0, i + 1);
        /*
        * check if that substring is part of dictionary or not?
        * If it is part of it, then add it in ans
        * make a call for remaining word
        */
        if(dict.contains(substr)) {
            solveWordBreak(dict, word.substring(i + 1), ans + substr + " ");
        }
    }
}

public static void wordBreak(HashSet<String> dict, String str) {
    solveWordBreak(dict, str, ans:"");
}

Run | Debug
public static void main(String[] args) {
    HashSet<String> set = new HashSet<>();
    set.add(e:"i");
    set.add(e:"like");
    set.add(e:"man");
    set.add(e:"go");
    set.add(e:"mango");
    set.add(e:"ice");
    set.add(e:"cream");
    set.add(e:"icecream");

    String word = "ilikemangoicecream";
    wordBreak(set, word);
}
```

```
i like man go ice cream
i like man go icecream
i like mango ice cream
i like mango icecream
```